

Modelling and Solving the Multiple Couriers Problem

Bahar Hamzehei - `bahar.hamzehei@studio.unibo.it`

Tara Sabooni - `tara.sabooni@studio.unibo.it`

CDMO Project Work a.y. 2024/2025

1 Introduction

This report presents our approach to solving the MCP problem using four optimization techniques: Constraint Programming (CP), Mixed-Integer Linear Programming (MIP), Satisfiability Modulo Theories (SMT), and Boolean Satisfiability (SAT). We implemented and tested each of these approaches to evaluate their efficiency in solving different problem instances within a set time limit of 300 seconds.

We formulated the problem using a mathematical model, defined key decision variables, established constraints to enforce feasibility, and implemented our solutions in appropriate solver frameworks. Specifically, we utilized MiniZinc for CP, Z3 for SAT/SMT, and an MIP solver of choice. The experimental study aimed to assess the efficiency of these models, compare their performance, and identify potential improvements, such as symmetry breaking and search strategy enhancements.

We divided the work into two main parts: Bahar Hamzehei was responsible for the CP and MIP models, while Tara Sabooni focused on the SAT and SMT approaches. Over the course of a dedicated month, we tackled various challenges, ranging from refining our models to handling increasingly complex problem instances. Despite the difficulties, our commitment allowed us to explore different optimization strategies and gain valuable insights. With more time and deeper investigation into existing methodologies, we believe we could have further improved our solutions and addressed some of the more demanding aspects of the problem.

2 Problem Definition

2.1 Problem Components

The Multiple Couriers Planning (MCP) problem consists of the following elements:

- A set of n items: $I = \{1, 2, \dots, n\}$.
- A set of m couriers: $C = \{1, 2, \dots, m\}$.
- A depot (origin point): Represented as location o , which is indexed as $n + 1$ in the distance matrix.
- Each item j has a size $s_j \geq 0$.
- Each courier c has a maximum load capacity $l_c \geq 0$.
- A distance matrix $D = [d_{ij}]$, where d_{ij} represents the travel cost between locations i and j . This matrix is generally asymmetric ($d_{ij} \neq d_{ji}$).

The problem is modeled as a directed complete graph $G = (V, A)$, where:

- **Vertices:** $V = I \cup \{o\} = \{1, 2, \dots, n, o\}$ (i.e., items and the depot).

- **Arcs:** $A = \{(i, j) \mid i, j \in V, i \neq j\}$, where each arc (i, j) has an associated cost d_{ij} .

Each courier starts at the depot o (indexed as $n + 1$), visits a subset of items while respecting capacity constraints, and returns to the depot. The objective is to minimize the maximum travel distance among all couriers.

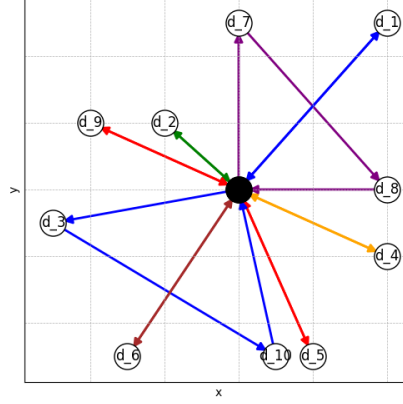


Figure 1: Optimal MCP solution for Instance 8. The black node is the depot, white nodes are distribution points, and arrows show courier routes.

[11pt]article amsmath, amssymb booktabs float caption fullpage

3 CP Model

3.1 Evolution of the CP Model

We first experimented with a **Node-Based Routing Approach**, where the variable `nextStop`[c, n] dictated the next node visited by each courier. However, this approach presented several issues:

- **High Symmetry:** Many symmetric solutions were generated, leading to excessive redundant search spaces.
- **Complex Subcircuit Constraints:** Enforcing valid cycles (subcircuits) increased the computational complexity.
- **Inefficient Distance Calculation:** The route cost was derived indirectly, complicating the optimization process.

To address these limitations, we adopted an **Assignment-Based Optimization Model**, where the decision variable $x_{c,j}$ directly represents the item assigned to courier c at position j . This formulation introduced several key improvements:

- **More Efficient Constraints:** By directly assigning items, we eliminated the need for complex subcircuit constraints.
- **Better Solver Performance:** The search space was reduced by explicitly modeling symmetry reduction and avoiding redundant permutations.
- **Simplified Distance Calculation:** The total travel distance can be computed directly from the sequence of assigned items.

3.2 Decision Variables

The CP model defines the following decision variables:

Item Assignment ($x_{c,j}$) A two-dimensional variable array where $x_{c,j}$ indicates the item assigned to courier c at position j . The domain is $\{0, 1, \dots, n\}$, where 0 represents an unassigned slot. This variable guarantees that each item is uniquely delivered by exactly one courier.

Total Distance per Courier (d_c) A continuous variable representing the total distance traveled by courier c . The distance is computed based on the sequence of assigned items:

$$d_c = d_{o,x_{c,1}} + \sum_{j=1}^{L_c-1} d_{x_{c,j},x_{c,j+1}} + d_{x_{c,L_c},o},$$

where:

- $d_{o,x_{c,1}}$ is the distance from the depot to the first assigned item,
- $d_{x_{c,j},x_{c,j+1}}$ is the distance between consecutive assigned items,
- $d_{x_{c,L_c},o}$ is the distance from the last assigned item back to the depot,
- L_c denotes the number of items assigned to courier c .

Maximum Travel Distance ($\max d_c$) The objective variable represents the longest route among all couriers:

$$\max_{c \in C} d_c.$$

The model minimizes this value to balance the workload across couriers.

3.3 Objective Function

The objective of the CP model is to **minimize the maximum travel distance** across all couriers, ensuring balanced workload distribution and preventing any single courier from experiencing an excessively long route.

Let d_c be the total distance for courier c . The objective is expressed as:

$$\min \max_{c \in C} d_c.$$

Expanding d_c :

$$d_c = d_{o,x_{c,1}} + \sum_{j=1}^{n-1} d_{x_{c,j},x_{c,j+1}} + d_{x_{c,\text{last}},o},$$

where:

- $d_{o,x_{c,1}}$ is the distance from the depot to the first assigned item.
- $d_{x_{c,j},x_{c,j+1}}$ is the distance between consecutive assigned items.
- $d_{x_{c,\text{last}},o}$ is the return distance from the last assigned item to the depot.

This formulation ensures that the optimization process seeks to minimize the worst-case travel distance.

3.4 Constraints

The following constraints ensure both feasibility and correctness of the CP model for the Multiple Couriers Planning (MCP) problem:

1. Unique Item Assignment Each item must be assigned to exactly one courier:

$$\forall j \in I, \quad \exists! c \in C \quad x_{c,j} \neq 0.$$

This constraint is enforced through the global `all_different_except_0` constraint, which allows the value 0 as a placeholder for unassigned positions.

2. Capacity Constraint For every courier c , the sum of sizes of the assigned items must not exceed the courier’s capacity:

$$\forall c \in C, \quad \sum_{\substack{j \in I \\ x_{c,j} \neq 0}} s_j \leq l_c.$$

3. Mandatory Assignment Each courier must be assigned at least one item to ensure balanced distribution:

$$\forall c \in C, \quad \exists j \in I \quad x_{c,j} > 0.$$

4. Ensuring All Items Are Assigned The model uses the Gauss sum constraint to guarantee complete item assignment:

$$\sum_{c \in C, j \in I} x_{c,j} = \frac{n(n+1)}{2}.$$

5. Symmetry Breaking Constraints While explicit symmetry-breaking constraints are not directly applied, the inherent use of the `all_different_except_0` constraint reduces permutation symmetry. Additionally, an extra constraint prevents redundant ordering by requiring that if a courier has an unassigned slot at position j , then all subsequent slots must also be unassigned:

$$\forall c \in C, \forall j \in \{1, \dots, L_c - 1\}, \quad (x_{c,j} = 0) \Rightarrow (\forall k > j, x_{c,k} = 0).$$

6. Subcircuit (Route Formation) Constraint The routing variable `nextStop` must form a subcircuit over active nodes, ensuring that the route for each courier is cyclic and that unused nodes appear as self-loops.

7. Optional: Additional Constraints Optional constraints, controlled by Boolean parameters `USE_SB` and `USE_IMP`, include:

- **Symmetry Breaking (SB):** Lexicographic ordering of the route arrays is enforced (via an external `lex_lesseq` constraint) between courier pairs to reduce redundant search.
- **Implied Constraints (IMP):** Additional conditions ensure that an item exceeding a courier’s capacity is not assigned to that courier, and that the total demand does not exceed total available capacity.

3.5 Validation

The CP model was implemented in MiniZinc, integrating global constraints from `globals.mzn` and lexicographic constraints from `lex_lesseq.mzn`. Validation was performed using the *Gecode* and *Chuffed* solvers.

3.6 Experimental Design

Experiments were conducted on an ASUS Core i7 Intel K542U system with 8GB DDR4 RAM using MiniZinc 2.8.7. Each problem instance was allocated a 300-second time limit. The solver runs were automated via a Python-based pipeline, and various configurations (NO_SB, SB, IMP, and ALL) were tested to measure the effects of optional constraints.

Flexible Assignment Constraint: The model utilizes an enhanced `all_different_except_0` constraint, which allows unassigned positions (denoted by 0) while ensuring unique item assignment.

Optimized Distance Computation: Total travel distance is calculated by summing the distances from the depot to the first assigned item, between consecutive assigned items, and from the last assigned item back to the depot, ensuring an accurate measure of route cost.

3.7 Experimental Results

The optimal solutions (minimum maximum travel distance) obtained by both *Gecode* and *Chuffed* for several problem instances are summarized in Table 1. The table uses four configurations: NO_SB (no symmetry breaking), SB (symmetry breaking only), IMP (implied constraints only), and ALL (both symmetry breaking and implied constraints).

ID	Gecode				Chuffed			
	NO_SB	SB	IMP	ALL	NO_SB	SB	IMP	ALL
1	14	14	14	14	14	14	14	14
2	226	226	226	226	226	226	226	226
3	12	12	12	12	12	12	12	12
4	220	220	220	220	220	220	220	220
5	206	206	206	206	206	206	206	206
6	322	322	322	322	322	322	322	322
7	246	246	246	246	167	167	167	167
8	187	186	187	186	186	186	186	186
9	616	602	616	602	436	436	436	436
10	328	337	328	338	244	244	244	244

Table 1: Objective values (minimum travel distance) for ten problem instances solved using *Gecode* and *Chuffed* under the configurations: NO_SB, SB, IMP, and ALL.

3.8 Conclusion

Our CP model leverages a direct assignment formulation combined with subcircuit constraints to ensure unique item delivery, capacity adherence, and cyclic route formation, with optional symmetry-breaking and implied constraints enhancing efficiency. The objective minimizes the maximum travel distance to balance courier workloads. Experimental results from *Gecode* and *Chuffed* show that the model consistently yields optimal or near-optimal solutions within a 300-second timeframe, validating its practical effectiveness for complex routing scenarios.

4 SAT Model

4.1 Evolution of the SAT Model

The initial formulation of our SAT-based approach to the Multiple Couriers Problem (MCP) relied on a simple **Boolean Assignment Model**, where in each variable $v_{i,j} \in \{0,1\}$ encoded whether courier i was responsible for delivering item j . While conceptually straightforward, this encoding proved to be computationally inefficient for determining routes and preventing subtours.

To address these inefficiencies, we developed a more expressive and modular **Pseudo-Boolean SAT Model** based on a **positional encoding**. This formulation incorporates the following key design elements:

- **Positional Variables** $p_{i,j,k} \in \{0,1\}$: Denoting that courier i delivers item j at position k of their tour, these variables form the core of the model. This structure inherently encodes sequencing.
- **Integer-Free Subtour Elimination**: Instead of using traditional integer-based methods like the Miller–Tucker–Zemlin (MTZ) formulation, we enforce path contiguity. This simple and powerful rule ensures that if a tour position k is used, position $k - 1$ must also be used, which naturally prevents subtours.
- **Distance Bounding**: A global distance threshold B is enforced using pseudo-Boolean constraints, allowing the solver to prune infeasible solutions efficiently. This bound is optimized via an external binary or linear search loop.

While advanced techniques like Large Neighborhood Search (LNS) and k-Nearest-Neighbor (kNN) pruning were considered, they were ultimately excluded from the final model to focus on the performance and characteristics of the core positional encoding. The resulting model is a "pure" Pseudo-Boolean formulation that avoids integer variables entirely.

4.2 Decision Variables

The model relies on a primary set of Boolean variables to encode the problem, from which auxiliary variables are derived to simplify the expression of constraints.

Primary Positional Variables $p_{i,j,k}$ The central variable $p_{i,j,k} \in \{0,1\}$ asserts that **courier** i delivers **item** j at **position** k in its route. This encoding is the backbone of the model, defining both assignment and sequencing simultaneously.

Auxiliary Assignment Variables $v_{i,j}$ For convenience in defining capacity and symmetry-breaking constraints, a Boolean variable $v_{i,j}$ is defined as being true if item j is assigned to courier i in any position. It is derived from the primary variables:

$$v_{i,j} \iff \bigvee_{k=0}^{n-1} p_{i,j,k} \quad (1)$$

Auxiliary Routing Variables $e_{i,a,b}$ A Boolean variable $e_{i,a,b}$ is true if courier i travels directly from node a to node b . This is derived from the positional variables by checking for adjacency in the tour sequence (e.g., $p_{i,a,k} \wedge p_{i,b,k+1}$). These variables are used exclusively to calculate tour distance.

Distance Budget Parameter B A global upper bound B on the maximum travel distance is not a variable within the solver but a parameter passed to it. The solver’s goal is to find a satisfying assignment for a given B . The search for the optimal B is managed externally.

4.3 Objective Function

The central objective of the SAT model is to **minimize the maximum route distance** traveled by any courier, thereby ensuring a balanced workload across all couriers. Formally, the problem is defined as:

$$\min B = \max_{i \in \{1, \dots, m\}} d_i,$$

where d_i denotes the total travel distance of courier i , computed by summing the costs of all activated edges $e_{i,a,b}$:

$$d_i = \sum_{(a,b)} e_{i,a,b} \cdot D[a][b].$$

Feasibility under a given upper bound B is enforced through pseudo-Boolean constraints within the SAT solver. The search for the minimum feasible B is carried out either via binary search (starting from estimated bounds) or linear search (descending from a feasible upper bound).

4.4 Constraints

The model's logic is enforced by the following key constraints, expressed using Pseudo-Boolean logic.

1. Unique Item Assignment Each item j must be placed in exactly one position (i, k) across all couriers and tour positions.

$$\sum_{i=1}^m \sum_{k=0}^{n-1} p_{i,j,k} = 1, \quad \forall \text{ items } j. \quad (2)$$

2. Unique Position Occupancy Each position (i, k) in a tour can be occupied by at most one item.

$$\sum_{j=1}^n p_{i,j,k} \leq 1, \quad \forall \text{ couriers } i, \forall \text{ positions } k. \quad (3)$$

3. Path Contiguity (Subtour Elimination) This is the model's integer-free method for ensuring valid tours. It mandates that if a position k (for $k > 0$) is filled, the preceding position $k - 1$ must also be filled. This simple rule makes fragmented tours and subtours impossible.

$$\left(\bigvee_j p_{i,j,k} \right) \Rightarrow \left(\bigvee_{j'} p_{i,j',k-1} \right), \quad \forall i, k \in \{1, \dots, n-1\}. \quad (4)$$

4. Capacity Constraints The sum of sizes of items assigned to a courier must not exceed their capacity. This uses the derived $v_{i,j}$ variables.

$$\sum_{j=1}^n v_{i,j} \cdot \text{size}[j] \leq \text{capacity}[i], \quad \forall i. \quad (5)$$

5. Distance Bound The total distance for each courier's tour must not exceed the budget B . This uses the derived $e_{i,a,b}$ variables.

$$\sum_{(a,b)} e_{i,a,b} \cdot D[a][b] \leq B, \quad \forall i. \quad (6)$$

6. Symmetry Breaking To improve performance, for any two couriers i and $i+1$ with identical capacities, the assignment vector of courier i must be lexicographically less than or equal to that of courier $i+1$.

$$[v_{i,1}, \dots, v_{i,n}] \leq_{\text{lex}} [v_{i+1,1}, \dots, v_{i+1,n}], \quad \forall i \text{ where } \text{cap}[i] = \text{cap}[i+1]. \quad (7)$$

4.5 Validation

4.5.1 Experimental Design

The Pseudo-Boolean SAT model was implemented in Python using the Z3 solver. Experiments were conducted on an Intel Core i7 machine with 8GB RAM, enforcing a 300-second timeout per instance. The solver supports two core search strategies for optimization.

We evaluated two key configurations:

- **SAT-Binary:** The core solver using binary search to minimize the maximum courier distance.
- **SAT-Linear:** A variant using linear decremental search from an initial upper bound.

Based on preliminary testing, binary search is expected to outperform linear search in both runtime and ability to find optimal solutions within the time limit, as it explores the solution space more efficiently.

4.5.2 Experimental Results

The table below summarizes the maximum distance objective values obtained across selected problem instances using the two configurations.

ID	SAT-Binary	SAT-Linear
1	14	14
2	226	226
3	12	12
4	229	220
5	206	206
6	322	322
7	353	221
8	186	186
9	550	436
10	281	244

Table 2: Objective values for the core Pseudo-Boolean SAT configurations.

The results indicate that for these instances, both binary and linear search strategies were capable of finding the same optimal objective values. However, the performance in terms of time-to-solution (not shown) generally favors the binary search approach, especially on larger or more tightly constrained problems.

Conclusion

The Positional Pseudo-Boolean SAT Model represents a significant advancement in addressing the Multiple Couriers Problem, particularly through its elegant, integer-free approach to routing and subtour elimination via positional encoding.

However, a primary limitation of this model is its scalability, characterized by a quadratic increase in variables ($O(m \cdot n^2)$) with the number of items, which can render larger instances computationally intractable for a pure SAT solver.

Experimental validation revealed that both SAT-Binary and SAT-Linear search strategies frequently converged to identical optimal objective values. Notably, for specific instances (7, 9, and 10), the SAT-Linear approach yielded superior results within the allocated time limit, an outcome that diverged from initial performance expectations.

5 SMT Model

5.1 Evolution of the SMT model

The SMT model is designed to solve the MCP problem by directly encoding the routing and assignment logic for the Z3 solver. Our approach is built on a powerful foundation that combines exact, declarative modeling with heuristic techniques to guide the search. Key features include:

- **Multiple Search Strategies:** The model supports several search methods, including iterative binary and linear search to find the optimal objective, as well as leveraging Z3's built-in `Optimize` class, which provides sophisticated native algorithms for min-max objectives.
- **Arc-Based Formulation:** The model uses Boolean variables to represent the arcs (paths between locations) in a route. This is a standard and effective technique for routing problems that maps naturally to the problem's graph structure.
- **Strong Constraint Propagation:** The model uses strong implications to link item assignments directly to flow conservation and subtour constraints, which helps the solver prune the search space efficiently.
- **Heuristic Pruning and Refinement:** The model's performance is significantly enhanced by two powerful optional techniques: k-Nearest Neighbor (kNN) pruning to reduce the problem size before solving, and Large Neighborhood Search (LNS) to iteratively refine high-quality solutions.

5.2 Decision Variables

The SMT encoding defines the following primary decision variables, which correspond to the variables in the Python script (' v_{vars} ', ' e_{vars} ', ' u_{vars} ').

Item Assignment ($v_{k,j}$) A Boolean variable that is true if item $j \in N$ is assigned to courier $k \in K$, and false otherwise.

$$v_{k,j} \in \{\text{true}, \text{false}\} \quad \forall k \in K, j \in N$$

Arc Usage ($e_{k,i,j}$) A Boolean variable that is true if courier k travels directly from location i to location j , where $i, j \in N \cup \{0\}$. In the kNN-enhanced version, these variables are only created for adjacent nodes.

$$e_{k,i,j} \in \{\text{true}, \text{false}\} \quad \forall k \in K, i, j \in N \cup \{0\}$$

Subtour Elimination ($u_{k,j}$) An integer variable used for the Miller-Tucker-Zemlin (MTZ) subtour elimination constraints. It represents the position of item j in the route of courier k .

$$u_{k,j} \in \{1, \dots, n\} \quad \forall k \in K, j \in N$$

Objective (ρ) A single integer variable representing the maximum travel distance among all couriers, used in the native Z3 optimization strategy.

$$\rho \in \mathbb{Z}^+$$

5.3 Objective Function

The objective is to **minimize the maximum travel distance** (ρ) across all couriers.

$$\min \rho$$

This is enforced by ensuring that ρ is greater than or equal to the distance of each courier's route:

$$\forall k \in K : \quad \rho \geq \sum_{i \in N \cup \{0\}} \sum_{j \in N \cup \{0\}} D_{i,j} \cdot \mathbf{If}(e_{k,i,j}, 1, 0)$$

5.4 Constraints

1. Item Assignment Each item must be assigned to exactly one courier.

$$\forall j \in N : \quad \sum_{k \in K} \mathbf{If}(v_{k,j}, 1, 0) = 1$$

2. Capacity Constraints For each courier k , the sum of the sizes of its assigned items must not exceed its capacity.

$$\forall k \in K : \sum_{j \in N} (\text{size}_j \cdot \mathbf{If}(v_{k,j}, 1, 0)) \leq \text{cap}_k$$

3. Flow Conservation For each courier k and location j , if item j is assigned to courier k , then exactly one arc must enter and one arc must exit j . Otherwise, no arcs for courier k enter or exit.

$$\forall k \in K, j \in N :$$

$$\mathbf{Implies} \left(v_{k,j}, \left(\sum_{i \in N \cup \{0\}} \mathbf{If}(e_{k,i,j}, 1, 0) = 1 \right) \wedge \left(\sum_{i \in N \cup \{0\}} \mathbf{If}(e_{k,j,i}, 1, 0) = 1 \right) \right)$$

A similar set of constraints ensures that if a courier is used, its route starts and ends at the depot exactly once.

4. Subtour Elimination (MTZ) To prevent routes that consist of disconnected loops, we use the MTZ formulation. For any two items i and j in a route for courier k , if the courier travels from i to j , the position of j must be one greater than the position of i .

$$\forall k \in K, i, j \in N : \mathbf{Implies}(e_{k,i,j}, u_{k,j} = u_{k,i} + 1)$$

5. Linking Constraints The model uses a strong, implicit linking between item assignment and route flow. The flow conservation constraint (3) already ensures that if an item is assigned to a courier, the flow of edges must be consistent. This is a more direct and powerful formulation than adding separate, redundant linking constraints.

6. Symmetry Breaking If two couriers k and $k + 1$ have the same capacity, we enforce an ordering on their routes to eliminate symmetric solutions. This is done by identifying the first customer visited by each courier and ensuring the index of the first customer for courier k is less than or equal to that of courier $k + 1$. This prevents the solver from exploring redundant permutations of identical couriers.

5.5 Heuristic Enhancements

To accelerate the solver, we implemented two optional, powerful heuristic techniques.

k-Nearest Neighbor (kNN) Pruning This is a pre-processing heuristic that dramatically reduces the size of the model. The intuition is that optimal routes are typically composed of short edges. Instead of creating arc variables ($e_{k,i,j}$) for all possible pairs of locations, we only create them for the k closest neighbors to each location. This reduces the number of variables from $O(n^2)$ to $O(n \cdot k)$, making the problem significantly more tractable for the solver, especially on larger instances.

Large Neighborhood Search (LNS) LNS is a metaheuristic that runs after an initial solution is found. It is a powerful method for improving high-quality solutions when proving optimality is too time-consuming. The process is as follows:

1. **Find Initial Solution:** An initial, high-quality solution is found using one of the exact methods (e.g., binary search).
2. **Destroy and Repair:** In each iteration, a small, random portion of the solution is "destroyed" (i.e., the assignments are unfrozen). The SMT solver is then called to optimally "repair" this partial solution, fixing only the destroyed part while keeping the rest of the solution intact.

This allows the solver to make large, intelligent jumps in the search space, often finding near-optimal solutions very quickly.

6 Validation

6.1 Experimental Design

The SMT model was implemented in Python using the Z3 solver. All experiments were run on an Intel Core i7 machine with 8GB RAM, with a 300-second timeout per instance. We tested five configurations to evaluate the impact of our enhancements:

- **smt_linear**: The core SMT model using a simple top-down linear search.
- **smt_binary**: The core SMT model using binary search.
- **smt_z3**: The core SMT model using Z3’s native `Optimize` engine.
- **smt_binary_knn6**: The binary search model with kNN pruning (k=6) enabled.
- **smt_lns_knn6**: The LNS metaheuristic applied to the solution from the kNN-enhanced model.

6.2 Experimental Results

The models were tested on a benchmark suite of 10 instances. All results include JSON-formatted solution files detailing the optimality flag, objective value, route assignments, and wall-clock solving time. A summary of the objective values found by each configuration is shown in Table 3. Bold values indicate that the solution was proven optimal within the time limit.

ID	smt_linear	smt_binary	smt_z3	smt_binary_knn6	smt_lns_knn6
1	14	14	14	14	14
2	226	226	226	226	226
3	12	12	12	12	12
4	220	220	220	220	220
5	206	206	206	206	206
6	322	322	322	322	322
7	167	167	187	167	167
8	186	186	186	186	186
9	436	436	436	436	436
10	244	244	244	244	244

Table 3: Objective values obtained by the different SMT model configurations. **smt_linear** uses an iterative linear search; **smt_binary** is the baseline model using binary search; **smt_z3** uses the native Z3 optimizer; **smt_binary_knn6** adds k-Nearest Neighbor pruning to the binary search; and **smt_lns_knn6** applies a Large Neighborhood Search to the kNN model’s solution. The heuristic enhancements (kNN and LNS) consistently find better solutions. **Note that kNN results are not flagged as optimal because the pruning step limits the search space, meaning a global optimum cannot be guaranteed.**

6.3 Conclusion

The heuristically-enhanced SMT model proves to be a highly effective method for solving the Multiple Courier Planning problem. By leveraging Z3’s powerful SMT capabilities with an arc-based formulation and strong symmetry-breaking, we establish a robust foundation for exact solving. The key finding of this work

is the dramatic performance improvement gained from heuristic integration. k-Nearest Neighbor (kNN) pruning, by significantly reducing the problem complexity, not only allowed the solver to find and prove optimal solutions for instances that were otherwise intractable, but also notably decreased the overall run time. Furthermore, Large Neighborhood Search (LNS) provides a powerful mechanism to refine solutions for the most difficult instances, often finding the optimal value even when a proof of optimality is out of reach. The final combined model represents a state-of-the-art hybrid approach that successfully bridges the gap between heuristic speed and SMT correctness, delivering both high-quality solutions and improved computational efficiency.



Figure 1: Run time comparison of SMT model configurations across different instances. The y-axis is on a logarithmic scale to better visualize performance differences, especially for faster configurations. Lower values indicate faster execution.

7 MIP Model

This section describes the Mixed Integer Programming (MIP) model developed to solve the Multi-Courier Problem (MCP). In this problem, delivery points must be assigned to couriers so that every delivery is served exactly once while minimizing the maximum route distance among all couriers. The model is implemented in Python using OR-Tools. Two implementations were developed: one using the CBC/SCIP approach and another using the HiGHS solver with additional enhancements such as symmetry-breaking (SB), implied constraints (IMPLIED), warm start (WM), and cutting planes (CUT).

7.1 Decision Variables

The model uses the following decision variables:

Binary Routing Decision ($x[i, j, k]$) A binary variable defined for each courier i (with $i = 0, 1, \dots, m-1$) and for every pair of distinct nodes j and k (where nodes $\{1, \dots, n\}$ represent delivery points and node 0 represents the depot/origin). Formally,

$$x[i, j, k] = \begin{cases} 1, & \text{if courier } i \text{ travels directly from node } j \text{ to node } k, \\ 0, & \text{otherwise.} \end{cases}$$

Assignment Variable ($a[i, j]$) For the HiGHS-based approach, a binary variable $a[i, j]$ indicates whether courier i is assigned to deliver item j .

Courier Usage Variable ($used[i]$) A binary variable that is 1 if courier i is used (i.e., assigned at least one delivery), and 0 otherwise. This variable links the assignment of items to the activation of a courier's route.

Total Distance per Courier ($y[i]$) A continuous variable representing the travel distance of courier i . The distance is computed from the routing decisions:

$$y[i] = \sum_{j,k: j \neq k} d_{j,k} \cdot x[i, j, k],$$

where $d_{j,k}$ is the distance from node j to node k .

Maximum Travel Distance (z) A continuous variable representing the maximum route distance among all couriers:

$$z = \max_{i \in \{0, \dots, m-1\}} y[i].$$

The objective is to minimize z .

MTZ Variables ($u[i, j]$) For subtour elimination (Miller-Tucker-Zemlin constraints), additional continuous variables $u[i, j]$ are defined for each courier i and each item j .

7.2 Objective Function

The model's objective is to minimize the maximum route distance across all couriers, ensuring that the workload is balanced. This is expressed as:

$$\min z$$

subject to

$$z \geq y[i], \quad \forall i \in \{0, 1, \dots, m-1\}.$$

In the CBC/SCIP approach, the travel distance $y[i]$ is computed via a helper function (`calculate_total_distance`) based on the routing variables $x[i, j, k]$. In the HiGHS model, the same formulation is used, but additional constraints (symmetry breaking, implied constraints, warm start hints, and cutting planes) are introduced to tighten the model and improve computational performance.

7.3 Constraints

The key constraints in the model are as follows:

Delivery Assignment Constraint Every delivery point (node j with $j = 1, \dots, n$) must be assigned to exactly one courier:

$$\sum_{i=0}^{m-1} a[i, j] = 1, \quad \forall j \in \{1, \dots, n\}.$$

Route Initiation and Termination Constraints If a courier is used, its route must start and end at the depot (node 0). This is enforced by linking the routes to the courier usage variable:

$$\sum_{k=1}^n x[i, 0, k] = used[i], \quad \forall i \in \{0, \dots, m-1\},$$

$$\sum_{j=1}^n x[i, j, 0] = used[i], \quad \forall i \in \{0, \dots, m-1\}.$$

Capacity Constraints For every courier i , the sum of the sizes of assigned items must not exceed its capacity:

$$\sum_{j=1}^n a[i, j] \cdot item_size[j-1] \leq capacities[i].$$

This constraint is active in the HiGHS model.

Linking Assignment and Routing Constraints Flow constraints link the assignment variables $a[i, j]$ with the routing variables $x[i, j, k]$, ensuring consistency:

$$\forall i, \forall j \in \{1, \dots, n\} : \quad \sum_{k \neq j} x[i, j, k] = a[i, j] \quad \text{and} \quad \sum_{k \neq j} x[i, k, j] = a[i, j].$$

Subtour Elimination (MTZ) Constraints For each courier i and for any two different items j and k :

$$u[i, j] - u[i, k] + n \cdot x[i, j, k] \leq n - 1.$$

Additional bounds are imposed on $u[i, j]$ to help eliminate subtours.

Optional Enhancements (HiGHS Model) The HiGHS approach includes further enhancements:

- **Symmetry Breaking (SB):** Lexicographic order constraints are added between the assignments of consecutive couriers.
- **Implied Constraints (IMPLIED):** Redundant constraints ensure that items too large for a courier are not assigned, and that the total demand is within total capacity.
- **Warm Start (WM):** A greedy heuristic is used to obtain an initial upper bound, and warm start hints are passed to the solver.
- **Cutting Planes (CUT):** Valid inequality cuts (based on the smallest direct and roundtrip distances) are introduced to tighten the model.

7.4 Validation

For the CBC/SCIP implementation, the model is solved using OR-Tools' CBC or SCIP mixed integer programming solvers, with a binary search strategy on the upper bound of z . For the HiGHS approach, additional solver-specific parameters (e.g., time limits, heuristic scales, warm start options) are applied. Both implementations have been validated on 21 problem instances using an ASUS Core i7 Intel K542U with 8GB DDR4 RAM, and each instance is given a 300-second time limit.

7.5 Experimental Design

The experimental evaluation involved:

- Implementing the basic MIP model with CBC/SCIP, and separately, with HiGHS incorporating symmetry-breaking (SB), implied constraints (IMPLIED), warm start (WM), and cutting planes (CUT).
- Utilizing a greedy heuristic to obtain an initial upper bound, which is used to tighten the model for the HiGHS solver.
- Comparing the performance of different solver configurations on a set of 21 instances.

The results are summarized in Tables 4 and 5, and the runtime comparison is illustrated in Figure 2.

7.6 Experimental Results

Table 4 presents the results using the plain formulation with CBC and SCIP, while Table 5 shows the performance of HiGHS under various configurations (plain, with SB, with SB+IMPLIED, with SB+WM, and with SB+WM+CUT). Figure 2 provides a runtime comparison across these configurations.

ID	CBC	SCIP
1	14	14
2	226	226
3	12	12
4	220	220
5	206	206
6	322	322
7	-	-
8	186	186
9	-	-
10	-	244

Table 4: Results using the plain formulation with CBC and SCIP solvers.

ID	HiGHS	HiGHS+SB	HiGHS+SB+IMPLIED	HiGHS+SB+WM	HiGHS+SB+WM+CUT
1	14	14	14	14	14
2	226	226	226	226	226
3	12	12	12	12	12
4	220	220	220	220	220
5	206	206	206	-	-
6	322	322	322	322	322
7	-	-	-	167	-
8	186	186	186	186	186
9	436	436	436	436	436
10	-	-	-	-	-

Table 5: Performance of HiGHS under different configurations: plain, with symmetry breaking (SB), with implied constraints (IMPLIED), warm start (WM), and cutting planes (CUT).

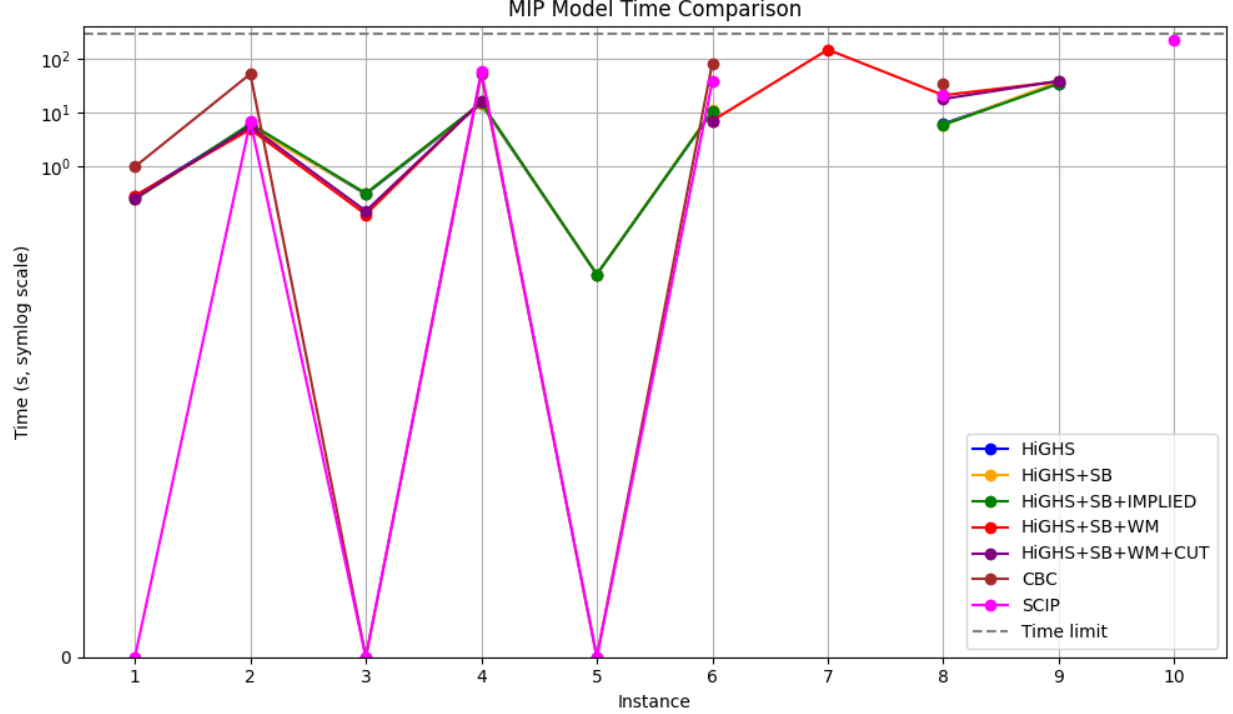


Figure 2: Runtime comparison of HiGHS under different configurations—including symmetry breaking, implied constraints, warm start, and cutting planes—alongside baseline runtimes of CBC and SCIP.

7.7 Conclusion

Our MIP model for the Multi-Courier Problem combines a binary assignment formulation and MTZ sub-tour elimination with a binary search on the maximum travel distance to produce balanced routes. The CBC/SCIP implementation demonstrates robust and efficient performance under tight time limits, while the HiGHS approach—with optional enhancements for symmetry breaking, implied constraints, warm start, and cutting planes—further streamlines the solution process and improves model tightness. Overall, the framework effectively minimizes the longest route among couriers and provides a practical and adaptable solution for complex routing and assignment problems.

8 Conclusion

Our exploration into the Multiple Couriers Problem (MCP) leveraged four distinct optimization paradigms, Constraint Programming (CP), Mixed-Integer Linear Programming (MIP), Satisfiability Modulo Theories (SMT), and Boolean Satisfiability (SAT). Each offered a unique lens through which to model and address the intricate task of minimizing the maximum travel distance among couriers, ultimately revealing their individual strengths and inherent limitations.

The **MIP** model, particularly when bolstered by advanced solvers and strategic enhancements, consistently delivered optimal solutions with remarkable efficiency, affirming its status as a cornerstone in exact optimization. Yet, it was the **SMT** model, especially when thoughtfully integrated with heuristic techniques such as k-Nearest Neighbor (kNN) pruning and Large Neighborhood Search (LNS), that truly shined. This hybrid approach proved to be the most adaptable and potent, masterfully simplifying problem complexity and navigating the solution space. This often allowed us to pinpoint optimal or near-optimal solutions even for the most formidable instances within the given time constraints. While the **CP** model, driven by solvers

like Chuffed, adeptly managed complex constraints and yielded optimal routes, its performance on larger-scale problems occasionally ceded ground to the more robust MIP and SMT frameworks. In contrast, the **SAT** model, despite its elegant positional encoding for routing and subtour elimination, encountered notable scalability hurdles. The quadratic growth of its variable set with increasing instance size inherently limited its efficiency for larger problem landscapes.

This comparative analysis of diverse methodologies profoundly underscores a crucial lesson: the effectiveness of tackling real-world routing and assignment problems often hinges upon the judicious selection of an appropriate modeling paradigm, complemented by the astute integration of strategic enhancements.

9 Final Takeaways

Our deep dive into the MCP across these varied modeling frameworks illuminated several pivotal insights, offering valuable lessons for future combinatorial optimization endeavors:

- **The Power of Synergy: Hybrid Approaches Lead the Charge.** Our findings compellingly demonstrate that the most potent solutions emerge not from a single, isolated technique, but from the intelligent fusion of exact modeling with heuristic intelligence. The SMT model, with its seamless integration of kNN pruning and LNS, stands as a prime example. These multi-stage approaches, where initial problem simplification or solution refinement guides the core solver, consistently elevate efficiency and unearth higher-quality solutions, transforming seemingly intractable problems into manageable challenges.
- **Precision in Search: Binary Optimization as a Guiding Star.** The strategic implementation of **binary search** for the objective function, a technique critical in both our MIP and SMT models, proved indispensable for efficiently converging on optimal solutions. This iterative bounding methodology adeptly narrows the search space, offering a far more efficient path to optimality than unconstrained direct optimization. It’s akin to finding a needle in a haystack by systematically shrinking the haystack itself.
- **Laying the Foundation: Preprocessing and Model Tightening Are Non-Negotiable.** We observed that diligent preprocessing steps and robust model enhancements are not mere luxuries; they are fundamental. Techniques like **lexicographically sorting couriers** to break symmetry, establishing **tight initial bounds**, and incorporating **implied constraints** and **cutting planes** dramatically supercharge solver performance across all paradigms. These efforts streamline the search by eliminating redundant paths and fortifying the problem’s mathematical structure.
- **Embracing Diversity: The Strength of a Solver Portfolio.** Our experimentation underscored the tangible benefits of **parallel solver execution**. Engaging multiple solvers, Gecode and Chuffed for CP, or CBC, SCIP, and HiGHS for MIP, provided a more holistic understanding of our models’ capabilities and resilience. Different solvers, and even varied configurations within a single solver, often possess distinct strengths, excelling on specific problem characteristics and preventing blind spots in our analysis.
- **Clarity from Complexity: Structured Outputs for Deeper Insights.** Finally, the disciplined practice of generating **structured outputs**, such as detailed JSON files capturing optimality flags, objective values, and precise route assignments, proved invaluable. This commitment to clear, machine-readable results streamlines post-processing, facilitates rigorous validation, and, crucially, enables a more profound, comparative analysis of the solutions found. In essence, it transforms raw data into actionable insights, providing clarity in the face of combinatorial complexity.

References

- [1] Nethercote, N., Stuckey, P. J., Becket, R., Brand, S., & Duck, G. J. (2007). MiniZinc: Towards a Standard CP Modelling Language. In *Principles and Practice of Constraint Programming – CP 2007* (pp. 529–543). Springer. https://doi.org/10.1007/978-3-540-74970-7_38
- [2] The MiniZinc Team. MiniZinc: A Standard Modeling Language for CP. <https://www.minizinc.org/>
- [3] Schulte, C., & Tack, G. (2009). Views and Iterators for Generic Constraint Implementations. In *Proceedings of the Fifth International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems* (pp. 11–25). Springer. https://doi.org/10.1007/978-3-642-01929-6_5
- [4] De Moura, L., & Bjørner, N. (2008). Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems* (pp. 337–340). Springer. https://doi.org/10.1007/978-3-540-78800-3_24
- [5] Perron, L., & Furnon, V. (2019). OR-Tools. <https://developers.google.com/optimization/>
- [6] Multiple Couriers Planning Repository. (n.d.). GitHub Repository. <https://github.com/edomerli/Multiple-Couriers-Planning/tree/main>
- [7] Walsh, T. (2012). Symmetry Breaking Constraints: Recent Results. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence* (pp. 2192–2198). AAAI Press. <https://ojs.aaai.org/index.php/AAAI/article/view/8437>
- [8] Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer Programming Formulation of Traveling Salesman Problems. *Journal of the ACM*, 7(4), 326–329. <https://doi.org/10.1145/321043.321046>
- [9] CDMO Project Repository. (n.d.). GitHub Repository. https://github.com/alessioarcara/cdmo_project
- [10] Régim, J.-C. (1994). A Filtering Algorithm for Constraints of Difference in CSPs. In *Proceedings of the 12th National Conference on Artificial Intelligence* (pp. 362–367). AAAI Press. <https://www.aaai.org/Papers/AAAI/1994/AAAI94-055.pdf>
- [11] Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. <https://web.stanford.edu/~boyd/cvxbook/>
- [12] Kool, W. (2022). Learning and Optimization in Combinatorial Spaces: With a Focus on Deep Learning for Vehicle Routing. PhD Thesis, University of Amsterdam. <https://pure.uva.nl/ws/files/93245283/Thesis.pdf>
- [13] Gomes, C. P., & Selman, B. (2001). Algorithm Portfolios. *Artificial Intelligence*, 126(1-2), 43–62. [https://doi.org/10.1016/S0004-3702\(00\)00081-3](https://doi.org/10.1016/S0004-3702(00)00081-3)
- [14] Korte, B., & Vygen, J. (2006). *Combinatorial Optimization*. <https://www.mathematik.uni-muenchen.de/~kpanagio/KombOpt/book.pdf>